

OTIMIZAÇÃO DE MODELOS DE APRENDIZADO SUPERVISIONADO

EAB067 - Aprendizado de Máquina

Atualizado em: 19 de maio de 2026

Iago Carvalho

Departamento de Ciências da Computação

Programa de Pós-Graduação em
Estatística Aplicada e Biometria



Como vimos nas aulas anteriores, algoritmos de aprendizado de máquina supervisionados utilizam dados **rotulados**

Além disso, também vimos que suas principais aplicações, os principais algoritmos de aprendizado e métodos de treinamento

Hoje veremos como treinar estes modelos de aprendizado supervisionado

Definição Formal

Dado um conjunto de treinamento $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$, gerado i.i.d. por uma distribuição de probabilidade conjunta desconhecida $P(x, y)$, o objetivo é induzir uma função $f : \mathcal{X} \rightarrow \mathcal{Y}$ a partir de um espaço de hipóteses $\mathcal{H}$.

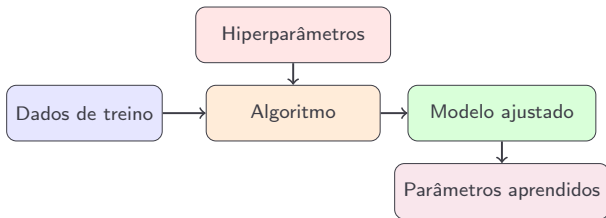
Componentes:

- $x \in \mathbb{R}^d$: Vetor de características (Features).
- $y \in \mathcal{Y}$: Variável alvo (Target).
- Se $\mathcal{Y} = \mathbb{R} \implies$ **Regressão**.
- Se $\mathcal{Y} = \{c_1, c_2, \dots, c_k\} \implies$ **Classificação**

OTIMIZAÇÃO DE HIPERPARÂMETROS

PARÂMETROS VS. HIPERPARÂMETROS

Antes de ajustar um modelo, é importante separar dois conceitos



Parâmetros

São aprendidos diretamente a partir dos dados durante o treinamento

- Coeficientes de uma regressão
- Pesos de uma rede neural
- Pontos de corte de uma árvore

Hiperparâmetros

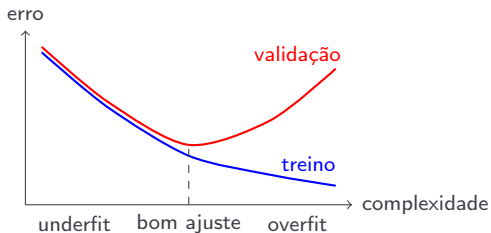
São definidos antes do treinamento e controlam como o modelo aprende

- Profundidade máxima de uma árvore
- Taxa de aprendizado
- Número de *clusters* no KNN

POR QUE OTIMIZAR HIPERPARÂMETROS?

Hiperparâmetros controlam diretamente o equilíbrio entre viés e variância

- Valores muito restritivos podem gerar *underfitting*
- Valores muito flexíveis podem gerar *overfitting*
- O objetivo é maximizar a performance de validação



Ideia central

Não avaliamos hiperparâmetros no conjunto de teste. O teste deve ser usado apenas uma vez, ao final, para estimar a performance final do modelo escolhido.

VALIDAÇÃO CRUZADA PARA ESCOLHA DE HIPERPARÂMETROS

Para cada combinação de hiperparâmetros, estimamos a performance média usando validação cruzada



1. Definimos um espaço de busca para os hiperparâmetros
2. Para cada configuração, executamos validação cruzada no conjunto de treino
3. Calculamos a média da métrica nos *folds*
4. Selecionamos a configuração com melhor desempenho médio
5. Reajustamos o modelo final usando todos os dados de treino
6. Avaliamos uma única vez no conjunto de teste

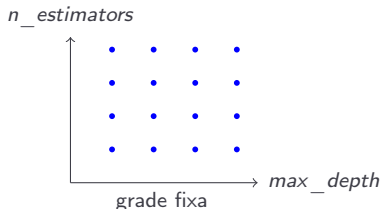
GRID SEARCH

O *grid search* testa exaustivamente todas as combinações de hiperparâmetros em uma grade pré-definida

Exemplo: Random Forest

- $n_estimators \in \{100, 300, 500\}$
- $max_depth \in \{3, 5, 10\}$
- $min_samples_split \in \{2, 5\}$

Total: $3 \times 3 \times 2 = 18$
configurações



Vantagens

Simples, reprodutível e fácil de interpretar

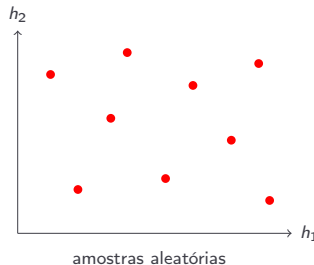
Desvantagens

Pode ser muito caro quando há muitos hiperparâmetros

RANDOM SEARCH

O *random search* sorteia configurações aleatórias dentro de distribuições ou intervalos definidos

- Não testa todas as combinações possíveis
- Permite explorar espaços de busca maiores
- Frequentemente encontra boas soluções com menor custo computacional
- É útil quando nem todos os hiperparâmetros têm a mesma importância



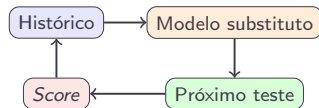
Intuição

Em muitos problemas, poucos hiperparâmetros dominam a performance. Sortear várias configurações pode ser mais eficiente do que testar uma grade rígida.

BUSCA BAYESIANA

A busca bayesiana usa os resultados anteriores para decidir quais configurações testar em seguida

- Constrói um modelo probabilístico da função objetivo
- Equilibra **exploração** e **exploração local**
- Tende a ser mais eficiente quando cada treinamento é caro
- É muito usada em modelos complexos



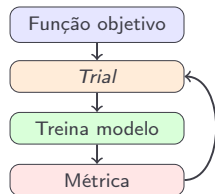
Função objetivo

Normalmente queremos maximizar uma métrica de validação ou minimizar uma perda média estimada por validação cruzada.

OPTUNA

Optuna é um pacote em Python para otimização automática de hiperparâmetros

- Define a busca como um problema de otimização
- O usuário escreve uma **função objetivo**
- Cada tentativa é chamada de *trial*
- O pacote sugere novos valores de hiperparâmetros
- Ao final, retorna a melhor configuração encontrada



Ideia central

Em vez de testar uma grade fixa, o Optuna aprende com os resultados anteriores e direciona a busca para regiões mais promissoras.

CUIDADOS NA OTIMIZAÇÃO DE HIPERPARÂMETROS

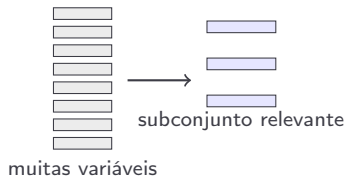
- **Vazamento de dados:** qualquer pré-processamento deve ocorrer dentro de cada *fold*
- **Escolha da métrica:** acurácia pode ser inadequada para dados desbalanceados
- **Custo computacional:** mais combinações nem sempre significam melhor generalização
- **Overfitting na validação:** testar muitas configurações pode superajustar ao conjunto de validação
- **Reprodutibilidade:** sempre controlar sementes aleatórias quando possível

SELEÇÃO DE VARIÁVEIS

POR QUE SELECIONAR VARIÁVEIS?

A seleção de variáveis busca manter apenas os atributos mais relevantes para a tarefa preditiva

- Reduz ruído e variância do modelo
- Pode melhorar a generalização
- Diminui custo computacional
- Facilita interpretação dos resultados



Atenção

Seleção de variáveis também deve ser feita dentro do processo de validação, evitando que informações do teste influenciem o treinamento

SELEÇÃO DE VARIÁVEIS NA PRÁTICA

Uma estratégia prática é combinar seleção de variáveis e otimização de hiperparâmetros dentro do mesmo *pipeline*

1. Separar treino e teste
2. Definir transformações de pré-processamento
3. Definir o método de seleção de variáveis
4. Definir o modelo e seu espaço de hiperparâmetros
5. Avaliar tudo por validação cruzada no treino
6. Treinar a configuração final e avaliar no teste

Regra de ouro

Tudo que aprende com os dados deve ser ajustado somente no conjunto de treino de cada *fold*.

TIPOS DE MÉTODOS DE SELEÇÃO

Podemos organizar os métodos de seleção em três grandes famílias

Filtros

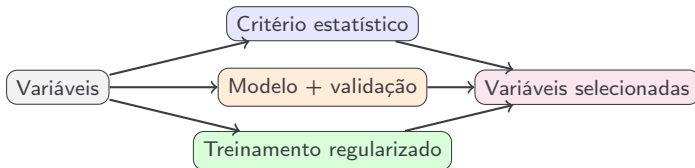
Avaliam variáveis antes do modelo preditivo

Wrappers

Testam subconjuntos usando um modelo

Embutidos

Selecionam variáveis durante o treinamento



A diferença central está em como a relevância das variáveis é estimada e no custo computacional envolvido

Métodos de filtro ranqueiam ou removem variáveis usando critérios estatísticos independentes do modelo final

- Correlação com a variável resposta
- Teste qui-quadrado para variáveis categóricas
- Remoção de variáveis com baixa variância
- Remoção de variáveis altamente correlacionadas entre si

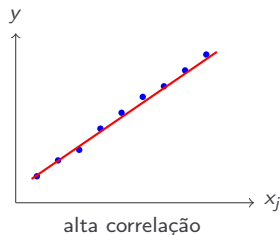
Resumo

São rápidos e simples, mas podem ignorar interações entre variáveis e o comportamento específico do modelo.

FILTRO POR CORRELAÇÃO COM A RESPOSTA

Mede a associação entre cada variável de entrada x_j e a variável resposta y

- Variáveis mais associadas a y recebem maior prioridade
- Para variáveis contínuas, é comum usar Pearson ou Spearman
- É rápido, mas captura principalmente relações lineares ou monotônicas



Quando usar?

Útil como triagem inicial em regressão ou quando buscamos relações simples entre variáveis e resposta.

FILTRO POR TESTE QUI-QUADRADO

Avalia se uma variável categórica está associada à classe observada

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

- O : frequência observada
- E : frequência esperada se não houvesse associação
- Quanto maior χ^2 , maior a evidência de associação
- Muito usado em classificação com variáveis categóricas ou contagens

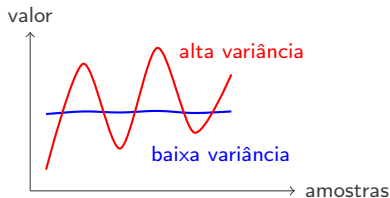
	Classe 0	Classe 1
$x_j = A$	30	5
$x_j = B$	10	25

Distribuições muito diferentes indicam associação.

FILTRO POR BAIXA VARIÂNCIA

Remove variáveis que quase não mudam entre as observações

- Variáveis quase constantes têm pouco poder discriminativo
- O critério não usa a variável resposta
- É útil para remover atributos triviais antes de métodos mais caros
- O limiar deve considerar a escala das variáveis



FILTRO POR CORRELAÇÃO ENTRE VARIÁVEIS

Remove variáveis redundantes que carregam praticamente a mesma informação

- Calculamos a correlação entre pares de variáveis de entrada
- Se $|cor(x_j, x_k)|$ é muito alto, uma das variáveis pode ser removida
- Reduz redundância e multicolinearidade
- Não garante que a variável mantida seja a mais preditiva para y

	x_1	x_2	x_3	x_4
x_1				
x_2				
x_3				
x_4				

pares em vermelho:
alta redundância

Métodos *wrapper* avaliam subconjuntos de variáveis treinando e validando modelos repetidamente

- **Forward selection:** começa sem variáveis e adiciona uma por vez
- **Backward elimination:** começa com todas e remove uma por vez
- **Recursive Feature Elimination (RFE):** remove iterativamente as variáveis menos importantes

Tradeoff

Costumam ter boa performance, mas são mais caros computacionalmente e podem superajustar se a validação não for bem planejada.

Métodos embutidos realizam seleção de variáveis como parte natural do treinamento do modelo

- **Lasso:** usa penalização L_1 e pode zerar coeficientes
- **Ridge:** usa penalização L_2 para reduzir os coeficientes de forma mais suave
- **Elastic Net:** combina penalizações L_1 e L_2
- **Gradient boosting:** fornece medidas de importância baseadas nas divisões usadas

Resumo

São eficientes porque combinam treinamento e seleção, mas a importância depende do tipo de modelo utilizado.

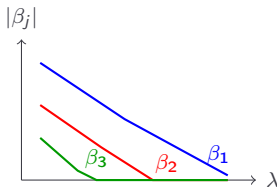
REGULARIZAÇÃO COMO SELEÇÃO DE VARIÁVEIS

Em modelos lineares, podemos controlar a complexidade por meio de penalizações nos coeficientes

$$\underbrace{\min_{\beta} \left[L(y, \hat{y}) + \lambda \sum_{j=1}^p |\beta_j| \right]}_{L_1}$$

$$\underbrace{\min_{\beta} \left[L(y, \hat{y}) + \lambda \sum_{j=1}^p \beta_j^2 \right]}_{L_2}$$

- λ controla a intensidade da penalização
- Quanto maior λ , maior a tendência de reduzir coeficientes



COMPARANDO L_1 E L_2

L_1 (Lasso)

- Soma valores absolutos
- Pode zerar coeficientes
- Produz modelos mais esparsos
- Útil para seleção de variáveis

L_2 (Ridge)

- Soma quadrados
- Reduz coeficientes continuamente
- Mantém quase todas as variáveis
- Útil com multicolinearidade

Elastic Net

Combina L_1 e L_2 , equilibrando seleção de variáveis e estabilidade dos coeficientes.

$$\min_{\beta} \left[L(y, \hat{y}) + \lambda \left(\alpha \sum_{j=1}^p |\beta_j| + (1 - \alpha) \sum_{j=1}^p \beta_j^2 \right) \right]$$

IMPORTÂNCIA POR PERMUTAÇÃO

A importância por permutação mede quanto a performance piora quando embaralhamos uma variável

1. Treinamos o modelo normalmente
2. Calculamos a performance em validação
3. Embaralhamos os valores de uma variável
4. Calculamos novamente a performance
5. Quanto maior a queda, maior a importância da variável

Vantagem

Pode ser aplicada a diferentes tipos de modelos, sendo uma estratégia de interpretação independente do algoritmo.

- Validação cruzada permite estimar melhor a capacidade de generalização
- Hiperparâmetros controlam o comportamento do treinamento
- *Grid search*, *random search* e busca bayesiana são estratégias comuns de otimização
 - Todos eles são implementados pelo Optuna
- Seleção de variáveis reduz dimensionalidade e pode melhorar interpretação
- Métodos filtro, *wrapper* e embutidos diferem em custo e dependência do modelo
- Dados de teste final deve ser preservado até o fim do processo